NEXUS
AI Tic-Tac-Toe
ENGINE

By DAKSH SIROHI 25BCE10909

For VITyarthi

Course: Fundamentals in AI & ML

Faculty: GARIMA JAIN

Slot: B14+B23+D21

1. The Problem and Its Significance

The primary objective of this project was to engineer an "unbeatable" Tic-Tac-Toe opponent within a robust, terminal-based application. While building a basic, two-player Tic-Tac-Toe game is a common programming exercise, the true challenge lies in implementing an artificial intelligence that can calculate optimal moves rather than relying on random placement or simple heuristics.

This matters because it serves as a practical, foundational application of game theory and decision-making algorithms. By solving a deterministic, perfect-information game like Tic-Tac-Toe, the project demonstrates how machines can map out future states and make mathematically optimal choices—concepts that scale up to more complex systems in computer science.

2. Approach to the Solution

To tackle this, the software was designed with a strict modular architecture to ensure scalability and ease of debugging. The application was split into five distinct components:

- **NexusRules:** The core engine handling game state, validation, and win/draw conditions.
- **NexusInterface:** The terminal UI manager responsible for parsing inputs and rendering the board.

- **NexusAI:** The neural decision module that houses the core algorithms for the computer's turns.
- **NexusLog:** A system auditor for tracking state changes and debugging.
- **Application Entry Point:** The main loop that routes data between the modules.

This separation of concerns meant that the AI logic could be tested and refined completely independently of how the game was displayed on the screen.

3. Key Decisions Made

- **Choosing Minimax over Heuristics:** Instead of writing a long list of if/else rules (e.g., "if center is open, take it"), I implemented the **Minimax algorithm**. This allows the AI

- to dynamically generate a game tree, looking ahead at all possible future moves to guarantee a win or a draw.
- **Implementing Alpha-Beta Pruning:** Recognizing that evaluating every single board state is computationally heavy, I added Alpha-Beta pruning to the Minimax algorithm. This optimization cuts off branches of the game tree that are mathematically proven to be worse than previously evaluated paths, drastically improving the engine's efficiency.
- **Creating a Dedicated Logging Module:** I decided early on to build NexusLog to trace the AI's decision pathways. This proved crucial for verifying that the engine was evaluating the game tree correctly.

4. Challenges Faced

- **Recursive Algorithm Complexity:** Conceptualizing and implementing the recursive logic for the Minimax algorithm was initially challenging. Ensuring the algorithm correctly alternated between "maximizing" the AI's score and "minimizing" the human's score required precise variable tracking.
- **Terminal Interface Management:** Creating a clean, refreshing terminal UI that didn't clutter the screen with past game states required careful handling of standard

output and clear screen commands, ensuring a smooth user experience.

- **Debugging AI Logic:** When the AI made a suboptimal move during early development, it was difficult to trace why. Building out the system auditor was a direct response to this challenge, allowing me to step through the AI's scoring process.

5. What Was Learned

This project solidified several core computer science concepts. I gained practical experience with tree traversal and recursive algorithms, specifically seeing how theoretical optimizations like Alpha-Beta pruning directly impact execution speed. Furthermore, the project reinforced the importance of modular software architecture; keeping the interface strictly separated from the game logic made debugging the complex AI algorithms significantly more manageable.